Nghiên cứu thuật toán quản lý bộ nhớ PagedAttention với mô hình Llama-3.2-1B-Instruct trên GPU NVIDIA GeForce GTX 1650

Tạ Đình Thiện — 20235839

Báo cáo tái hiện — Kwon et al., SOSP '23

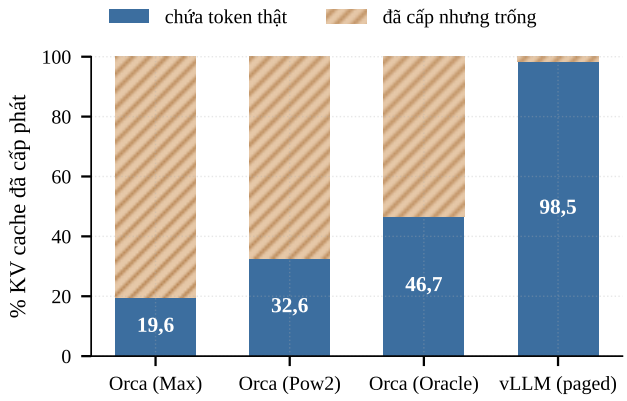
Tóm tắt

Kích thước batch của một hệ thống phục vụ mô hình ngôn ngữ lớn (LLM) bị giới hạn bởi bộ nhớ dành cho KV cache, và do đó cách quản lý vùng nhớ này quyết định throughput. Kwon và cộng sự đề xuất PagedAttention, quản lý KV cache theo các block kích thước cố định được cấp phát theo nhu cầu như bộ nhớ ảo của hệ điều hành, và báo cáo khả năng chịu tải cao hơn 2–4× so với các hệ thống đặt chỗ liên tục. Bài này kiểm chứng lại kết luận đó trong một môi trường gần như đối lập: một GPU laptop GTX 1650 4 GB không có tensor core và mô hình Llama-3.2-1B. Hai engine suy luận được viết từ đầu bằng CUDA, chỉ khác nhau ở cơ chế cấp phát KV cache — một bản đặt chỗ liên tục kiểu Orca với buddy allocator và một bản phân trang kiểu vLLM. Trên trace ShareGPT, bản phân trang duy trì được tốc độ request cao hơn 1,79× so với Orca (Max) và 1,13× so với Orca (Oracle); 98,5% KV cache đã cấp phát của nó chứa token thật, so với 19,6–46,7% ở các biến thể Orca. Lợi thế nhỏ hơn bài gốc vì trên phần cứng này throughput của bản phân trang chạm trần tính toán trước khi chạm trần bộ nhớ. Các thí nghiệm ablation cho thấy kernel attention phân trang không chậm hơn bản liền mạch cùng cài đặt, block size 16 nằm trong vùng tối ưu, và — ngược với bài gốc — swap KV cache sang bộ nhớ host rẻ hơn tính lại 10–180× ở mọi block size vì GPU yếu về sức tính.

1 Giới thiệu

Phục vụ LLM là một bài toán bộ nhớ nhiều hơn là một bài toán tính toán. Ở giai đoạn sinh token, mỗi bước decode phải đọc toàn bộ trọng số mô hình từ DRAM của GPU bất kể đang xử lý bao nhiêu request, nên gộp thêm request vào cùng một batch gần như không tốn thêm thời gian. Thứ giới hạn kích thước batch là KV cache — trạng thái key/value mà mỗi request phải giữ cho mọi token trong ngữ cảnh của nó. Với Llama-3.2-1B, mỗi token chiếm 32 KiB KV cache; trên GPU 4 GB dùng trong bài này, sau khi trọng số chiếm 2,30 GiB, phần còn lại chỉ chứa được khoảng 27 518 token (860 MiB).

Các hệ thống phục vụ như Orca [2] và FasterTransformer [3] lưu KV cache của mỗi request trong một vùng nhớ liền mạch, đặt trước cho độ dài tối đa có thể. Vì độ dài đầu ra không biết trước, phần lớn vùng đặt trước bị bỏ trống trong suốt vòng đời request; bộ cấp phát còn làm tròn kích thước và để lại các mảnh vụn giữa các vùng. Kwon et al. [1] đo được chỉ 20,4–38,2% KV cache trong các hệ thống này thực sự chứa token. Họ đề xuất PagedAttention: chia KV cache thành các block nhỏ cố định, cấp phát theo nhu cầu và tra qua một block table, như phân trang trong bộ nhớ ảo. Hình 1 cho thấy phép đo tương ứng trên hệ thống trong bài: 19,6%, 32,6% và 46,7% ở ba biến thể Orca, so với 98,5% ở bản phân trang — rất sát con số 20,4/26,8/38,2/96,3% của bài gốc.



Hình 1. Tỉ lệ KV cache đã cấp phát thực sự chứa token, trung bình theo thời gian trên trace ShareGPT ở 2,0 req/s. Số liệu tự đo từ telemetry của hai bộ cấp phát; thí nghiệm tương ứng với Hình 2 của [1].

Các kết quả của [1] được đo trên GPU A100 với các mô hình OPT 13B–175B — nơi bộ nhớ khan hiếm nhưng sức tính dồi dào. Bài này hỏi: lợi ích của phân trang còn lại bao nhiêu khi cả hai đều khan hiếm? GTX 1650 không có tensor core, đạt khoảng 2,7 TFLOP/s ở bf16 và 192 GB/s băng thông bộ nhớ — thấp hơn A100 khoảng hai bậc độ lớn về tính toán. Đóng góp của bài gồm:

- Hai engine suy luận viết tay bằng CUDA cho Llama-3.2-1B, **giống hệt nhau trừ cơ chế cấp phát KV cache**; sự giống nhau được kiểm tra tự động ở mỗi lần build.
- Tái hiện các thí nghiệm lấy mẫu cơ bản (§6.2), chatbot (§6.5) và ba ablation (§7.1–7.3) của [1] trên 142 lượt chạy.

- Đo trực tiếp những đại lượng mà [1] chỉ minh hoạ: tỉ lệ bộ nhớ đã cấp chứa token theo thời gian, và số lần cấp phát thất bại do phân mảnh ngoài.
- Hai phát hiện khác bài gốc: lợi thế của phân trang bị chặn bởi trần tính toán của GPU, và swap rẻ hơn tính lại ở mọi block size.

2 Bối cảnh

2.1 Sinh văn bản tự hồi quy và KV cache

Một request gồm hai pha. Pha *prefill* xử lý toàn bộ prompt trong một lượt và là phép nhân ma trận lớn, bị giới hạn bởi sức tính. Pha *decode* sinh từng token một; mỗi bước cần key và value của mọi token trước đó, nên chúng được lưu lại thành KV cache thay vì tính lại. Pha decode bị giới hạn bởi băng thông bộ nhớ: với Llama-3.2-1B, mỗi bước phải đọc 2,30 GiB trọng số, tức tối thiểu khoảng 12 ms trên 192 GB/s, bất kể batch có bao nhiêu request. Vì vậy số request xử lý được đồng thời — và throughput — tỉ lệ gần như trực tiếp với số request mà KV cache chứa được.

2.2 Batching ở mức vòng lặp

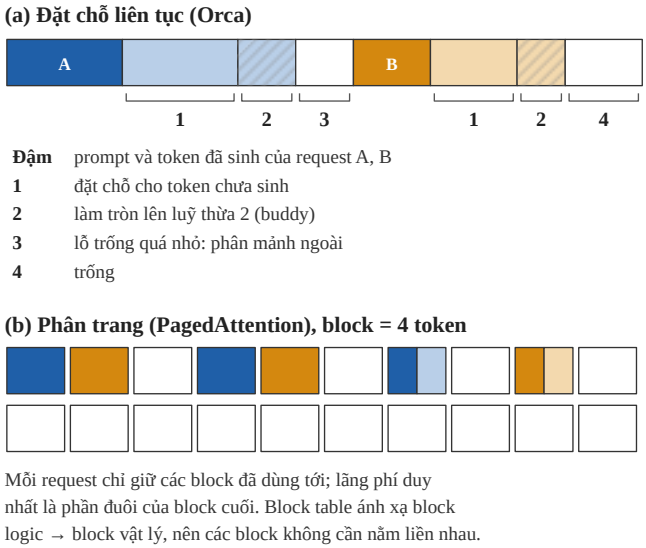
Orca [2] lập lịch theo từng bước decode thay vì theo từng batch: request mới được đưa vào ngay khi có chỗ, và request xong được giải phóng ngay. Cả hai engine trong bài đều dùng cơ chế này, nên khác biệt giữa chúng chỉ nằm ở việc *có chỗ* hay không.

2.3 Đặt chỗ liên tục và ba loại lãng phí

Khi KV cache của một request phải nằm liền mạch, hệ thống phải đặt chỗ cho toàn bộ độ dài tương lai ngay khi nhận request (Hình 2a). Điều đó gây ra ba loại lãng phí [1]: (i) *đặt chỗ* — vùng dành cho token chưa sinh; (ii) *phân mảnh trong* — phần làm tròn của bộ cấp phát và phần đặt dư so với độ dài thật; (iii) *phân mảnh ngoài* — khoảng trống giữa các vùng, đủ về tổng nhưng không đủ liền mạch cho request mới.

2.4 PagedAttention

PagedAttention [1] chia KV cache thành các block cố định (mặc định 16 token). Mỗi request có một block table ánh xạ block logic sang block vật lý; block mới chỉ được cấp khi block cuối đã đầy (Hình 2b). Lãng phí duy nhất là phần chưa đầy của block cuối, và vì mọi block cùng kích thước nên không có phân mảnh ngoài. Đổi lại, hệ thống không còn bảo đảm một request đã nhận sẽ có đủ chỗ đến cuối: khi pool cạn giữa chừng, nó phải *preempt* một request và sau đó phục hồi bằng cách tính lại KV cache hoặc swap nó qua bộ nhớ host.



Hình 2. Sơ đồ minh hoạ hai cơ chế. (a) Đặt chỗ liên tục: mỗi request giữ một vùng cho toàn bộ độ dài tối đa. (b) Phân trang: request chỉ giữ các block đã dùng tới, nằm rải rác trong pool.

3 Thiết kế và cài đặt

3.1 Engine chung

Hai engine phục vụ Llama-3.2-1B-Instruct [6] ở bf16 (16 layer, hidden 2048, 32 query head, 8 KV head theo GQA, head dim 64). Mọi kernel — RMSNorm, RoPE, attention, SwiGLU, lấy mẫu — viết tay bằng CUDA; riêng các phép nhân ma trận dùng cuBLAS. Lập lịch theo FCFS ở mức vòng lặp; mọi prompt đang chờ được prefill gộp trong một lượt tối đa 3072 token, và decode chạy tối đa 384 request đồng thời. Cửa sổ ngữ cảnh là 2048 token (1024 cho prompt, 1024 cho đầu ra), bằng độ dài tối đa của OPT trong [1]. Engine nhận request qua stdin và chỉ báo lại số token, không gửi văn bản ra ngoài.

3.2 Baseline kiểu Orca

Mã nguồn của Orca không công khai, nên giống [1], Orca được cài đặt lại với ba chính sách đặt chỗ: **Orca (Max)** đặt chỗ cho 2048 token; **Orca (Pow2)** đặt prompt cộng độ dài đầu ra làm tròn lên luỹ thừa 2 kế tiếp; **Orca (Oracle)** đặt đúng prompt cộng độ dài đầu ra thật. Pow2 và Oracle cần biết trước độ dài đầu ra nên không cài đặt được trong thực tế; chúng là cận trên. Như [1] giả định, vùng nhớ được cấp bởi buddy allocator, nên mọi đặt chỗ còn được làm tròn lên luỹ thừa 2. Một request chỉ được nhận khi tìm được một vùng liền mạch đủ lớn; mỗi lần thất bại trong khi tổng bộ nhớ trống vẫn đủ được đếm là một lần phân mảnh ngoài.

3.3 Bản phân trang kiểu vLLM

Pool được chia thành block 16 token. Một request được nhận khi prompt của nó vừa pool; block cho phần sinh ra được cấp dần trong lúc decode. Khi pool cạn, engine preempt request đến sau cùng, giải phóng toàn bộ block

của nó và — theo §4.5 của [1] — ngừng nhận request mới cho đến khi mọi request bị preempt hoàn thành. Hai chính sách phục hồi được cài đặt: *tính lại* (prefill lại prompt cùng các token đã sinh) và *swap* (chép các block sang bộ nhớ host pinned rồi chép lại khi có chỗ).

3.4 Bảo đảm so sánh công bằng

Ba cơ chế giữ cho phép so sánh chỉ có một biến số. Thứ nhất, các hằng số dùng chung của hai engine nằm trong một khối cấu hình phải giống nhau từng byte, và script build từ chối build nếu hai khối lệch nhau. Thứ hai, không ai chọn kích thước KV cache: mỗi engine cấp phát trọng số và bộ đệm tạm trước, rồi lấy toàn bộ VRAM còn trống (trừ 128 MiB dự phòng) làm KV cache. Kết quả là 27 518 token cho bản liền mạch và 27 390 token cho bản phân trang; chênh lệch 0,47% chính là bộ nhớ của block table. Thứ ba, cả hai dùng cùng file workload, cùng seed và cùng chuỗi thời điểm đến; EOS bị bỏ qua để mỗi request sinh đúng số token đầu ra ghi trong trace.

4 Đánh giá

4.1 Thiết lập thí nghiệm

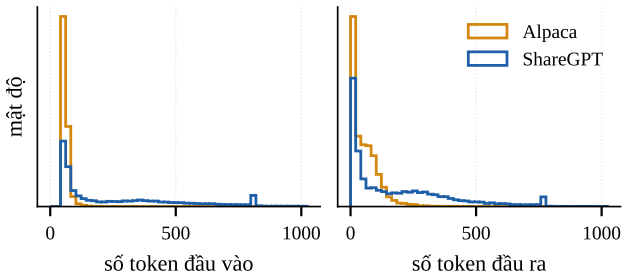
Phần cứng và mô hình. NVIDIA GTX 1650 laptop (Turing, 14 SM, 3,63 GiB VRAM khả dụng, 192 GB/s, giới hạn công suất 30 W, không tensor core), CUDA 12.0, Llama-3.2-1B-Instruct ở bf16.

Workload. Như [1], độ dài đầu vào và đầu ra của mỗi request lấy từ các cặp hỏi–đáp thật của ShareGPT [4] và Alpaca [5], đã tokenize bằng tokenizer của Llama (Bảng 1, Hình 3). Mẫu dài hơn cửa sổ ngữ cảnh bị loại thay vì cắt ngắn, để không tạo ra một cụm nhân tạo tại độ dài tối đa. Vì cửa sổ ngắn hơn, ShareGPT ở đây chỉ dài hơn Alpaca 4,3× ở đầu vào và 3,4× ở đầu ra, so với 8,4× và 5,8× trong [1]. Workload chatbot (§4.5) được dựng như §6.5 của [1]: mỗi lượt hội thoại gửi lại toàn bộ lịch sử, cắt về 1024 token cuối. Thời điểm đến tuân theo phân phối Poisson.

Bảng 1. Độ dài (token) của ba workload sau khi lọc, 20 000 mẫu mỗi bộ.

Workload	Vào tb	p50	p99	Ra tb	p50	p99
Alpaca	61	58	117	55	40	247
ShareGPT	266	152	866	186	113	769
ShareGPT hội thoại	777	948	1024	309	285	769

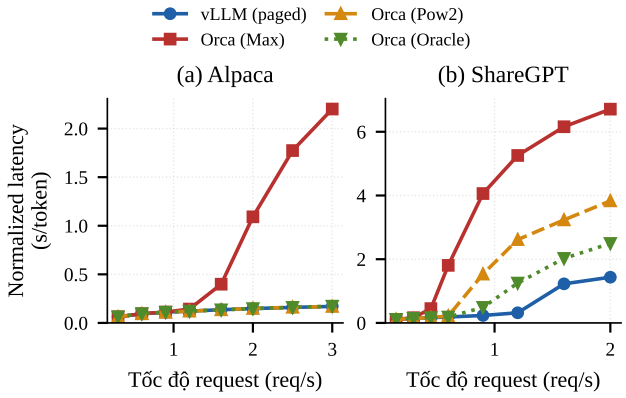
Chỉ số. Như [1], chỉ số chính là *normalized latency*: thời gian end-to-end của mỗi request chia cho số token đầu ra, lấy trung bình trên mọi request. Khả năng chịu tải của một hệ thống được tóm tắt bằng *throughput tối đa duy trì được*: số request hoàn thành mỗi giây cao nhất đạt được trên lưới tốc độ đến; khi quá tải, tốc độ hoàn thành ngừng tăng theo tốc độ đến và dừng ở đúng giá trị này. Ngoài ra còn đo *time to first token* (TTFT), vốn không bị pha loãng bởi độ dài đầu ra.



Hình 3. Phân phối độ dài đầu vào và đầu ra của ShareGPT và Alpaca.

Quy trình. Mỗi điểm đo là một lượt chạy 200 request trên lưới 8 tốc độ đến cho mỗi workload. Thay cho trace 1 giờ của [1], sự hội tụ của mỗi lượt chạy được kiểm tra bằng cách so normalized latency của nửa đầu với nửa sau. Các biến thể được chạy xen kẽ, nghỉ 60 s giữa hai lượt, để GPU laptop bị giới hạn nhiệt không thiên vị biến thể chạy sau; xung nhịp SM cuối mỗi lượt nằm trong 1 620–1 785 MHz.

4.2 Lấy mẫu cơ bản



Hình 4. Normalized latency theo tốc độ request trên (a) Alpaca và (b) ShareGPT. Số liệu tự đo; thí nghiệm tương ứng với Hình 12 của [1].

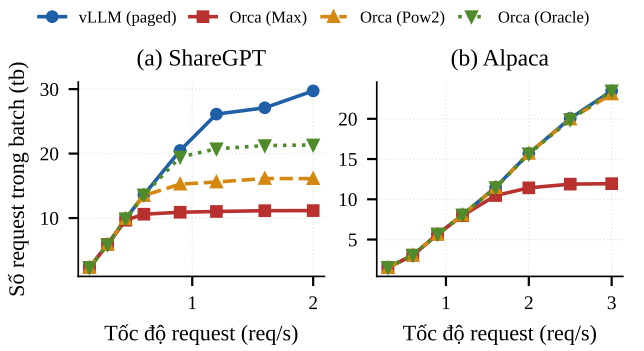
Hình 4 có cùng hình dạng như Hình 12 của [1]: normalized latency tăng chậm, rồi tăng vọt khi tốc độ đến vượt khả năng phục vụ và hàng đợi bắt đầu dồn lại. Trên ShareGPT, bản phân trang duy trì được 0,78 req/s, cao hơn Orca (Max) 1,79×, Orca (Pow2) 1,38× và Orca (Oracle) 1,13× (Bảng 2). Bài gốc báo cáo 2,7–8× so với Orca (Max) và 1,7–2,7× so với Orca (Oracle). Thứ tự giữa các hệ thống được giữ nguyên, nhưng biên độ nhỏ hơn; §6 phân tích nguyên nhân.

Bảng 2. Throughput tối đa duy trì được (req/s) và lợi thế của bản phân trang. Dấu ≥ đánh dấu giá trị vẫn đang tăng ở tốc độ đến cao nhất đã đo, tức là cận dưới; khi cả hai hệ thống đều chưa bão hoà, tỉ số không xác định được (—).

Workload	Hệ thống	req/s	paged / hệ thống
Alpaca	vLLM (paged)	≥ 2,50	
	Orca (Max)	1,51	≥ 1,66×
	Orca (Pow2)	≥ 2,48	—
	Orca (Oracle)	≥ 2,49	—
ShareGPT	vLLM (paged)	0,78	
	Orca (Max)	0,43	1,79×
	Orca (Pow2)	0,56	1,38×
	Orca (Oracle)	0,69	1,13×
ShareGPT hội thoại	vLLM (paged)	≥ 0,28	
	Orca (Max)	0,24	≥ 1,19×
	Orca (Pow2)	0,24	≥ 1,16×
	Orca (Oracle)	0,25	≥ 1,14×

Trên Alpaca, ba biến thể Orca (Pow2), Orca (Oracle) và bản phân trang gần như trùng nhau và chưa bão hoà ở 3,0 req/s, trong khi Orca (Max) bão hoà ở 1,51 req/s. Điều này khớp với nhận xét của [1] rằng khi chuỗi ngắn, lợi thế so với Pow2 và Oracle kém rõ rệt: một request Alpaca trung bình chỉ dùng khoảng 116 token, nên mọi chính sách đặt chỗ hợp lý đều để pool còn thừa.

4.3 Vì sao phân trang phục vụ được nhiều hơn



Hình 5. Số request trung bình trong batch theo tốc độ request. Số liệu tự đo; thí nghiệm tương ứng với Hình 13 của [1].

Kích thước batch. Hình 5 cho thấy mắt xích nhân quả. Ở 2,0 req/s trên ShareGPT, bản phân trang chạy trung bình 29,7 request đồng thời, gấp 1,39× Orca (Oracle) và 2,66× Orca (Max) (bài gốc: 2,2× và 4,3×). Orca (Max) không bao giờ vượt quá 13 request: $\lfloor 27\,518 / 2048 \rfloor = 13$, đúng bằng số vùng 2048 token vừa trong pool.

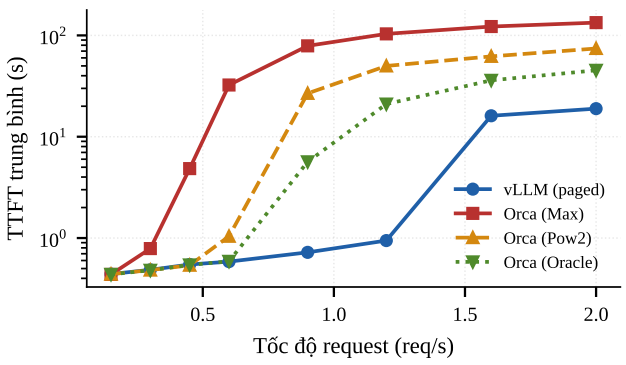
Bộ nhớ. Hình 1 đo nguyên nhân trực tiếp. Orca (Max) chỉ dùng 19,6% vùng đã cấp; phần còn lại là chỗ đặt trước cho token chưa sinh. Pow2 và Oracle đặt chỗ chính xác hơn nhưng vẫn lãng phí hơn một nửa, vì vùng dành cho token thứ 400 nằm trống trong suốt thời gian request mới sinh đến token thứ 10, và vì buddy allocator làm tròn lên luỹ thừa 2. Bảng 3 cho thấy cái giá thứ hai của việc đặt chỗ chính xác: các vùng kích thước khác nhau làm pool vụn ra, và Pow2 thất bại tới 1 191 lần, Oracle 689 lần trong một lượt chạy vì phân mảnh ngoài. Orca (Max) không bị phân mảnh ngoài vì mọi vùng cùng kích thước 2048, và bản phân trang không bị vì mọi block cùng kích thước 16. Trên Alpaca, không biến thể nào bị phân mảnh ngoài: hầu hết đặt chỗ rơi vào cùng một bậc luỹ thừa 2.

Bảng 3. Số lần cấp phát thất bại do phân mảnh ngoài (tổng bộ nhớ trống đủ nhưng không có vùng liền mạch đủ lớn) trên ShareGPT, theo tốc độ request.

req/s	vLLM	Orca (Max)	Orca (Pow2)	Orca (Oracle)
0,15	0	0	0	0
0,3	0	0	0	0
0,45	0	0	0	0
0,6	0	0	119	0
0,9	0	0	761	173
1,2	0	0	1 191	376
1,6	0	0	836	606
2,0	0	0	858	689

Cái giá của over-subscription. Bản phân trang nhận request mà không bảo đảm chỗ cho đầu ra, nên đôi khi phải preempt. Preempt chỉ xuất hiện khi hệ thống đã ở sát hoặc vượt ngưỡng bão hoà: trên ShareGPT từ 1,6 req/s, trên workload chatbot từ 0,27 req/s, và nhiều nhất 8,5% số request. Thời gian tính lại chiếm tối đa 4,4% thời gian chạy. Mọi request bị preempt đều hoàn thành đủ số token.

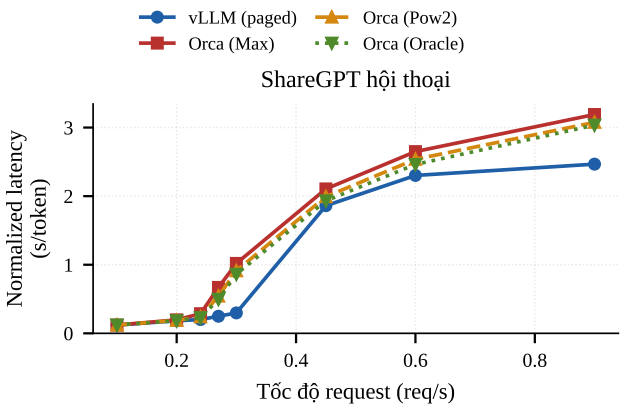
4.4 Thời gian đến token đầu tiên



Hình 6. TTFT trung bình trên ShareGPT, trục tung logarit.

Normalized latency chia thời gian chờ cho số token đầu ra, nên che bớt trải nghiệm của người dùng. Hình 6 cho thấy TTFT, tức thời gian nằm hàng đợi cộng prefill. Ở 1,2 req/s — tốc độ cao nhất mà hàng đợi của bản phân trang còn gần rỗng — bản phân trang trả token đầu tiên sau 0,94 s, Orca (Oracle) sau 21,0 s và Orca (Max) sau 104 s (110×). Ở tốc độ thấp nhất, cả bốn đường trùng nhau, xác nhận rằng khác biệt ở các tốc độ cao hơn đến từ bộ nhớ chứ không từ tốc độ tính toán.

4.5 Chatbot



Hình 7. Normalized latency theo tốc độ request trên workload chatbot. Số liệu tự đo; thí nghiệm tương ứng với Hình 17 của [1].

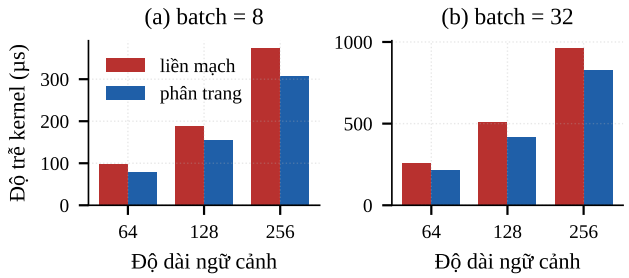
Hình 7 tái hiện Hình 17 của [1]. Prompt của workload này rất dài (trung vị 948 token), và bản phân trang duy trì được ≥ 0,28 req/s so với 0,24 của Orca (Max) (≥ 1,19×); bài gốc báo cáo 2×. Khác biệt lớn hơn nằm sau điểm bão hoà: ở 0,3 req/s, normalized latency của bản phân trang là 0,30 s/token, thấp hơn Orca (Max) 3,4×.

[1] quan sát rằng ba biến thể Orca hành xử như nhau ở đây, vì prompt của hầu hết request chạm trần 1024 token và buddy allocator làm tròn mọi đặt chỗ lên 2048. Ở đây chỉ Pow2 và Oracle trùng nhau, còn Max tệ hơn. Nguyên nhân tính được từ workload: chỉ 46% prompt chạm trần 1024, và với số còn lại Pow2 và Oracle vẫn đưa được khoảng một phần ba số đặt chỗ xuống mức ≤ 1024 token, trong khi Max làm tròn 100% đặt chỗ lên 2048.

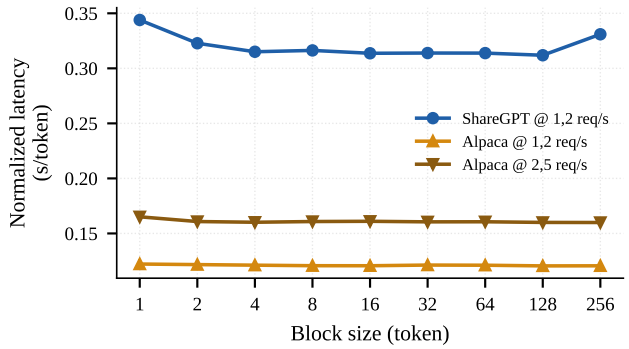
5 Ablation

5.1 Kernel attention

Truy cập qua block table thêm một bước tra bảng và làm KV cache của một request nằm rải rác. Để đo riêng chi phí này, kernel attention decode được chạy tách khỏi phần còn lại của mô hình, với cùng độ dài ngữ cảnh, cùng batch và cùng dữ liệu; hai kernel cho ra kết quả giống nhau từng bit. Trái với mức chậm hơn 20–26% trong [1], kernel phân trang ở đây chạy bằng 0,81–0,86× thời gian của kernel liền mạch (Hình 8). Khác biệt nằm ở baseline: [1] so với kernel của FasterTransformer, được tối ưu riêng rất sâu, còn ở đây hai kernel cùng một cài đặt, chỉ khác cách tính địa chỉ. Offset trong một block vừa với số nguyên 32 bit và vòng lặp trong chạy trên block size là hằng số lúc biên dịch, trong khi bản liền mạch phải dùng offset 64 bit. Kết luận đúng không phải "phân trang nhanh hơn" mà là chi phí tra bảng nhỏ hơn khác biệt giữa hai cách viết cùng một kernel.



Hình 8. Độ trễ kernel attention decode, phân trang so với liền mạch, cùng số byte KV cache. Số liệu tự đo; thí nghiệm tương ứng với Hình 18a của [1].



Hình 9. Normalized latency theo block size, với tốc độ request cố định. Số liệu tự đo; thí nghiệm tương ứng với Hình 18b của [1].

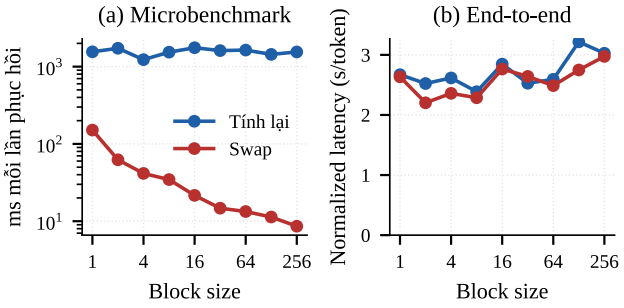
5.2 Block size

Block nhỏ làm mỗi lần đọc KV cache lấy được ít byte liên tiếp; block lớn làm phần trống của block cuối lớn lên. Hình 9 quét block size từ 1 đến 256 với số slot cố định 64 để block table vừa VRAM ở block size 1. Trên ShareGPT, block size từ 16 đến 128 cho kết quả tốt nhất và cách nhau không quá 0,6%, đúng như [1]; block 1 chậm hơn 9,6% và block 256 chậm hơn 5,5% so với 16. Trên Alpaca, [1] thấy block lớn làm hiệu năng giảm mạnh; ở đây mọi block size nằm trong khoảng 2,5% của nhau. Phân mảnh trong vẫn xảy ra — ở block 256, hơn một nửa bộ nhớ đã cấp cho Alpaca bị trống — nhưng chỉ biến thành độ trễ khi pool chật, và với chuỗi ngắn của Alpaca pool không bao giờ đầy quá khoảng 17% ở các tốc độ đã đo.

5.3 Tính lại so với swap

Workload chatbot được chạy ở 1,2 req/s, nơi pool bị dùng gần hết, với từng chính sách phục hồi và từng block size, mỗi điểm có 9–29 lần preempt. Xu hướng khớp với [1]: chi phí tính lại gần như không phụ thuộc block size (1,23–1,76 s), còn chi phí swap giảm khi block lớn lên, từ 151 ms ở block 1 xuống 8,6 ms ở block 256, vì block nhỏ tạo ra nhiều lượt truyền PCIe nhỏ. Nhưng điểm giao nhau không xuất hiện: [1] thấy tính lại hiệu quả hơn ở block nhỏ và không bao giờ đắt hơn swap quá 20%, còn ở đây tính lại đắt hơn swap 10–180× ở mọi block size. Lý do là

phía tính toán: tính lại phải prefill lại hơn một nghìn token trên một GPU không có tensor core, trong khi KV cache của một request chỉ khoảng 64 MiB và đi qua PCIe trong vài chục ms. Vì preempt hiếm, ảnh hưởng end-to-end nhỏ: normalized latency của mọi cấu hình nằm trong 2,2–3,2 s/token (Hình 10b).



Hình 10. (a) Chi phí một lần phục hồi request bị preempt theo block size. (b) Normalized latency end-to-end trên workload chatbot ở 1,2 req/s. Số liệu tự đo; thí nghiệm tương ứng với Hình 19 của [1].

6 Thảo luận

Vì sao lợi thế nhỏ hơn bài gốc. Phân trang tăng số request vừa trong bộ nhớ, nhưng điều đó chỉ thành throughput khi bộ nhớ là nút thắt. Trên ShareGPT ở 2,0 req/s, bản phân trang đã bão hoà trong khi KV cache của nó chỉ được dùng trung bình 43%: nút thắt đã chuyển sang sức tính, chủ yếu là prefill — phép nhân ma trận chạy trên lõi FP32 ở khoảng 2,7 TFLOP/s. Trên A100 với mô hình 13B trở lên, sức tính dư thừa hơn nhiều so với bộ nhớ, nên mỗi request thêm vào batch đều chuyển thành throughput và lợi thế của phân trang hiện ra đầy đủ. Workload chatbot, với prompt dài nhất và prefill nặng nhất, có lợi thế nhỏ nhất, nhất quán với giải thích này. Nói cách khác, lợi ích của phân trang bị chặn trên bởi tỉ số giữa trần tính toán của GPU và trần bộ nhớ của baseline.

Ngoài phạm vi. Bài chưa đánh giá các tình huống chia sẻ KV cache của [1]: parallel sampling, beam search và tiền tố dùng chung (§6.3–6.4). Engine phân trang đã hỗ trợ chia sẻ block với copy-on-write, nhưng các thí nghiệm tương ứng chưa được hoàn tất. Bài cũng không có baseline FasterTransformer.

7 Kết luận

Trên một GPU 4 GB không có tensor core, PagedAttention giữ được kết luận định tính của [1]: nó loại bỏ gần như toàn bộ lãng phí KV cache (98,5% bộ nhớ đã cấp chứa token, so với 19,6–46,7% ở các baseline), tăng số request xử lý đồng thời, không có phân mảnh ngoài, và nhờ đó duy trì được tốc độ request cao hơn 1,79× so với Orca (Max) trên ShareGPT. Biên độ nhỏ hơn bài gốc vì trên phần cứng yếu về tính toán, bộ nhớ thôi là nút thắt trước khi phân trang dùng hết lợi thế của nó. Chi phí của phân trang — tra block table, preempt khi over-subscribe — đo được là nhỏ. Kết quả cũng cho thấy lựa chọn chính sách phục hồi phụ thuộc phần cứng: trên GPU yếu về tính toán, swap luôn thắng tính lại.

Tài liệu tham khảo

1. Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP '23*.
2. Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI '22*.
3. NVIDIA. FasterTransformer. https://github.com/NVIDIA/FasterTransformer.
4. ShareGPT Team. 2023. https://sharegpt.com/.
5. Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca.
6. Meta AI. 2024. Llama 3.2: Lightweight Models. https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct.